

MODEL AND RESPONSES

Output styles

Adapt Claude Code for uses beyond software engineering

Output styles change how Claude responds, not what Claude knows. They modify the system prompt to set role, tone, and output format. Use one when you keep re-prompting for the same voice or format every turn, or when you want Claude to act as something other than a software engineer.

A custom output style adds your instructions to the system prompt and lets you choose whether to keep Claude Code's built-in software engineering instructions. Keep them when you're changing how Claude communicates but still coding, like always answering with a diagram. Leave them out when Claude isn't doing software engineering at all, like a writing assistant or data analyst.

For instructions about your project, conventions, or codebase, use [CLAUDE.md](#) instead.

Built-in output styles

Claude Code's **Default** output style is the existing system prompt, designed to help you complete software engineering tasks efficiently.

There are three additional built-in output styles:

- **Proactive:** Claude executes immediately, makes reasonable assumptions instead of pausing for routine decisions, and prefers action over planning. This is stronger autonomous-execution guidance than [auto mode](#) applies, and it works without changing your permission mode, so you still see permission prompts before tools run.
- **Explanatory:** Provides educational “Insights” in between helping you complete software engineering tasks. Helps you understand implementation choices and codebase patterns.
- **Learning:** Collaborative, learn-by-doing mode where Claude will not only share “Insights” while coding, but also ask you to contribute small, strategic pieces of code yourself. Claude Code will add `TODO(human)` markers in your code for you to implement.

Change your output style

Run `/config` and select **Output style** to pick a style from a menu. Your selection is saved to `.claude/settings.local.json` at the local project level.

❗ The standalone `/output-style` command was deprecated in v2.1.73 and removed in v2.1.91. Use `/config` or edit the `outputStyle` setting directly.

To set a style without the menu, edit the `outputStyle` field directly in a settings file:

```
{
  "outputStyle": "Explanatory"
}
```

Output style is part of the system prompt, which Claude Code reads once at session start. Changes take effect after `/clear` or a new session. See [How Claude Code uses prompt caching](#) for what an output style change does to the cache.

Create a custom output style

A custom output style is a Markdown file: frontmatter for metadata, then the instructions to add to the system prompt.

1 Create a Markdown file

Save it at one of three levels. The file name becomes the style name unless you set `name` in the frontmatter.

- User: `~/.claude/output-styles`
- Project: `.claude/output-styles`
- Managed policy: `.claude/output-styles` inside the managed settings directory

Project output styles load from every `.claude/output-styles/` between the working directory and the repository root. As of v2.1.178, when more than one of these nested directories defines a style with the same name, Claude Code uses the one closest to the working directory.

2 Add frontmatter and instructions

Decide whether to keep Claude Code's software engineering instructions. Set `keep-coding-instructions: true` if you're changing how Claude communicates but still want it coding the same way. Leave it out if Claude won't be doing software engineering.

This example leads every explanation with a diagram while keeping Claude's coding behavior:

```
---
name: Diagrams first
description: Lead every explanation with a diagram
keep-coding-instructions: true
---
```

When explaining code, architecture, or data flow, start with a Mermaid diagram showing the structure, then explain in prose.

Diagram conventions

Use `'flowchart TD'` for control flow and `'sequenceDiagram'` for request paths. Keep diagrams under 15 nodes.

3 Switch to your style

Run `/config` and select your style under **Output style**. It takes effect after `/clear` or the next time you start a session.

Plugins can also ship output styles in an `output-styles/` directory.

Frontmatter

Output style files support these frontmatter fields:

Frontmatter	Purpose	Default
<code>name</code>	Name of the output style, if not the file name	Inherits from file name
<code>description</code>	Description of the output style, shown in the <code>/config</code> picker	None

Frontmatter	Purpose	Default
<code>keep-coding-instructions</code>	Keep Claude Code's built-in software engineering instructions	<code>false</code>
<code>force-for-plugin</code>	Plugin output styles only: apply this style automatically whenever the plugin is enabled, without requiring users to select it. Overrides the user's <code>outputStyle</code> setting. If multiple enabled plugins set this, Claude Code uses the first one loaded.	<code>false</code>

How output styles work

Output styles directly modify Claude Code's system prompt.

- All output styles have their own custom instructions added to the end of the system prompt.
- All output styles trigger reminders for Claude to adhere to the output style instructions during the conversation.
- Custom output styles leave out Claude Code's built-in software engineering instructions, such as how to scope changes, write comments, and verify work, unless `keep-coding-instructions` is set to `true`.

Token usage depends on the style. Adding instructions to the system prompt increases input tokens, though prompt caching reduces this cost after the first request in a session. The built-in Explanatory and Learning styles produce longer responses than Default by design, which increases output tokens. For custom styles, output token usage depends on what your instructions tell Claude to produce.

Comparisons to related features

Several features customize how Claude Code behaves. Output styles modify the system prompt directly and apply to every response. The others add instructions without changing the default system prompt, or scope them to a specific task.

Feature	How it works	Use it when
Output styles	Modifies the system prompt	You want a different role, tone, or default response format every turn
<u>CLAUDE.md</u>	Adds a user message after the system prompt	Claude should always know your project conventions and codebase context
<code>--append-system-</code>	Appends to the system prompt without	You want a one-off addition for a single

Feature	How it works	Use it when
<code>prompt</code>	removing anything	invocation
<u>Agents</u>	Runs a subagent with its own system prompt, model, and tools	You want a separately scoped helper for a focused task
<u>Skills</u>	Loads task-specific instructions when invoked or relevant	You have a reusable workflow

Related resources

- Settings: where the `outputStyle` field lives and how settings precedence works
- Permission modes: how the Proactive style compares to auto mode
- Plugins: package and distribute output styles alongside skills, hooks, and agents
- Debug your configuration: diagnose why an output style isn't taking effect